



10-423 / 10-623 / 10-723 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

State Space Models + Hybrid Models

Aran Nayebi & Matt Gormley
Lecture 21

Reminders

- **Project Proposal**
 - Due: this Fri Apr 3 at 11:59pm
- **Quiz 5**
 - Mon, Apr 6 in class (Lectures 16-20)
- **HW623**
 - Only for students registered in 10-623 & 10-723
 - Due: Mon, Apr 20 at 11:59pm

Motivation

- Transformers are slow at test time: they require a KV cache that grows linearly in size with the sequence length
- State space models (SSMs) are fast at test time: they only hold a fixed size hidden state in memory (like RNNs)
- But we'll see that SSMs can also be trained efficiently with the right tricks
- As well, they elegantly transition between different granularities of representation for the input (e.g. sound at 16KHz vs. 8KHz)

Motivation

- <https://www.isattentionallyouneed.com/>

Is Attention All You Need?



Current Status: Yes

Time Remaining: 631d 22h 55m 11s

Proposition:

On January 1, 2027, a Transformer-like model will continue to hold the state-of-the-art position in most benchmarked tasks in natural language processing.

Motivation

- <https://www.isattentionallyyouneed.com/>

Is Attention All You Need?

For the Motion

Jonathan Frankle
@jefrankle
Harvard Professor
Chief Scientist Mosaic ML



Against the Motion

Sasha Rush
@srush_nlp
Cornell Professor
Research Scientist Hugging Face 🤗



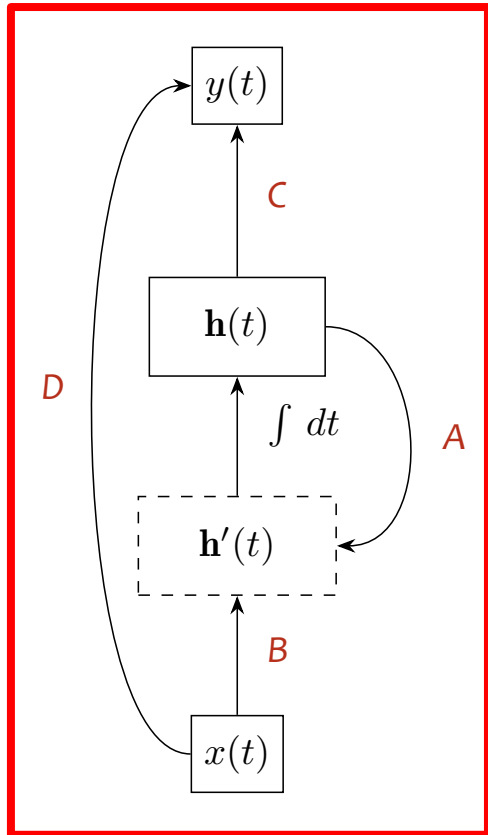
Wager

The wager is for donation of equity in Mosaic ML or Hugging Face to a charity of the winner's choice. Details to come.

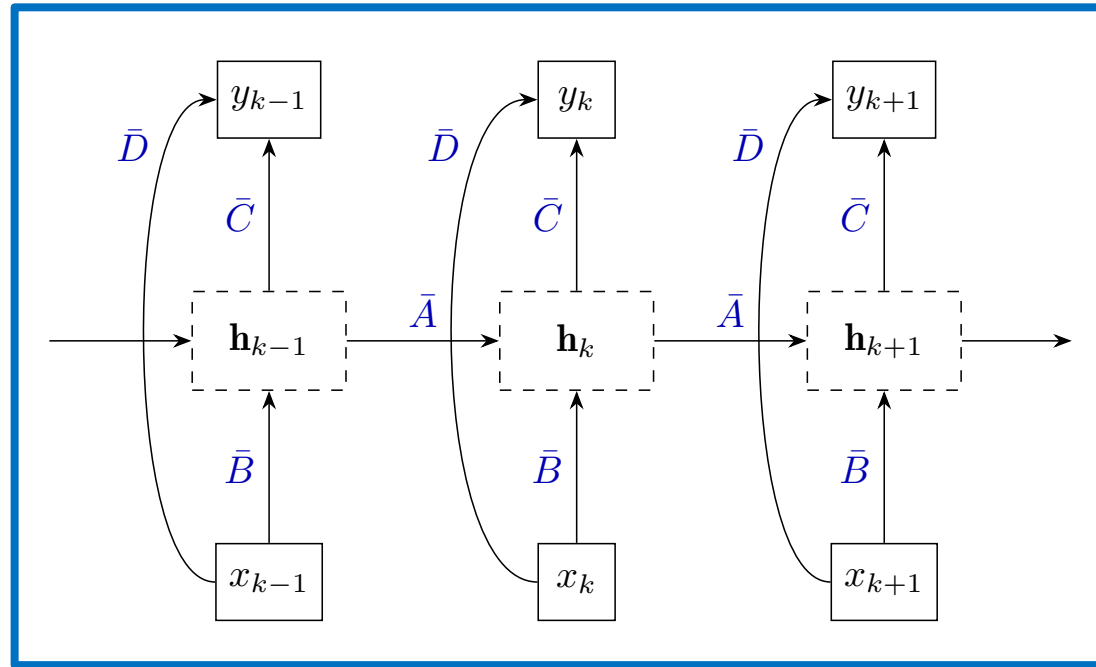
STATE SPACE MODEL (SSM)

Three Representations of a State Space Model

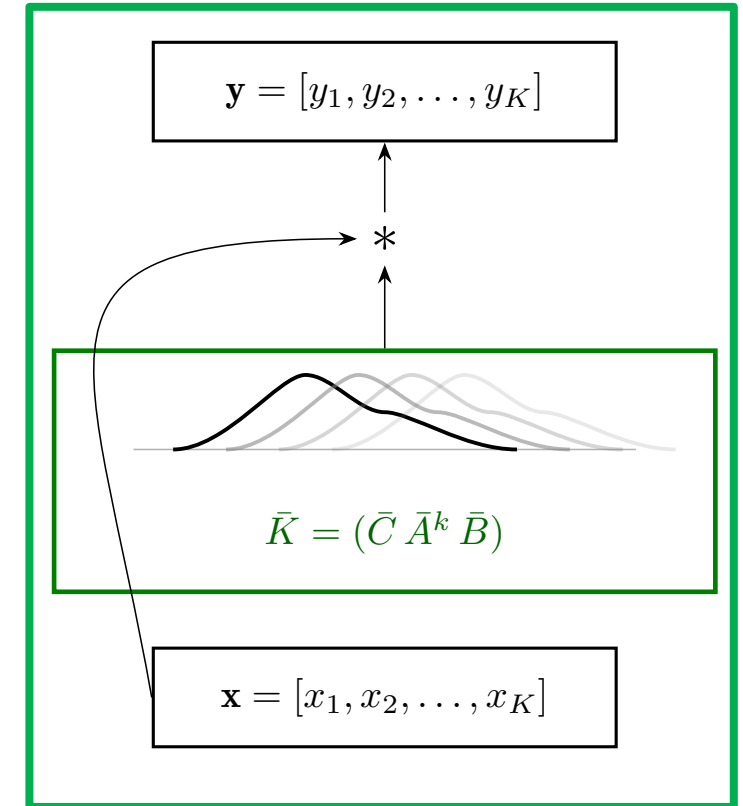
Continuous



Recurrent



Convolutional



SSM: 1D Continuous Representation

$$\frac{\partial \mathbf{h}(t)}{\partial t} = \mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t)$$

$$y(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}x(t)$$

$$x(t) \in \mathbb{R}$$

$$y(t) \in \mathbb{R}$$

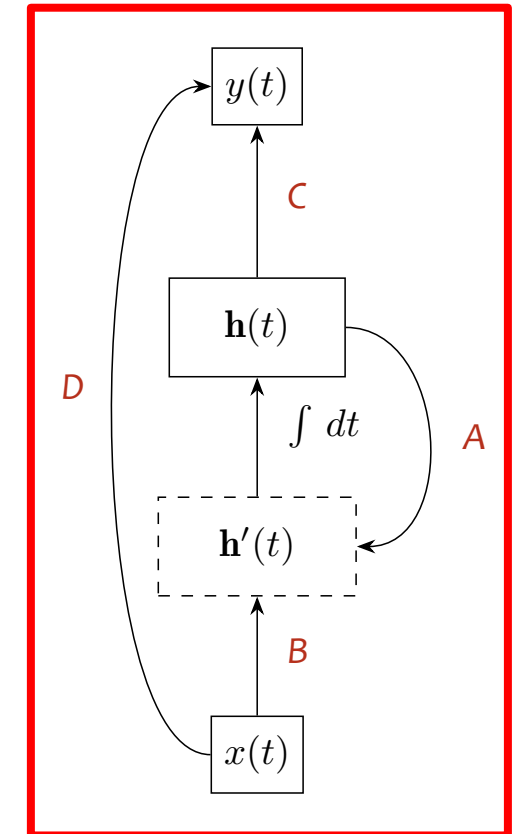
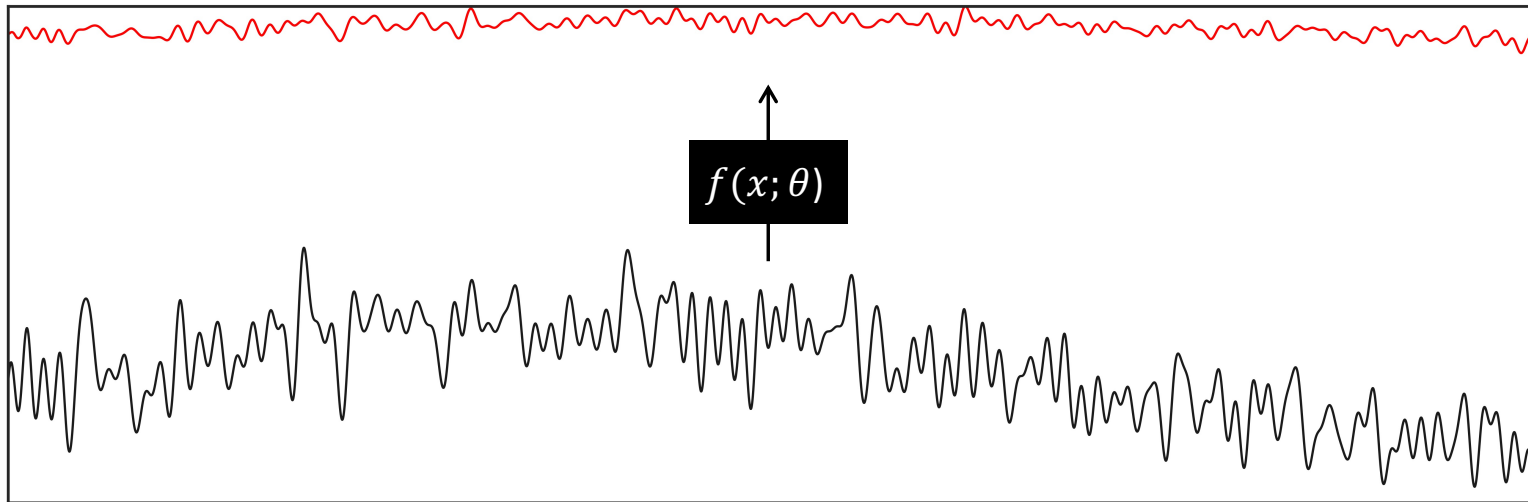
$$\mathbf{h}(t) \in \mathbb{R}^N$$

$$\mathbf{A} \in \mathbb{R}^{N \times N}$$

$$\mathbf{B} \in \mathbb{R}^{N \times 1}$$

$$\mathbf{C} \in \mathbb{R}^{1 \times N}$$

$$\mathbf{D} \in \mathbb{R}^{1 \times 1}$$



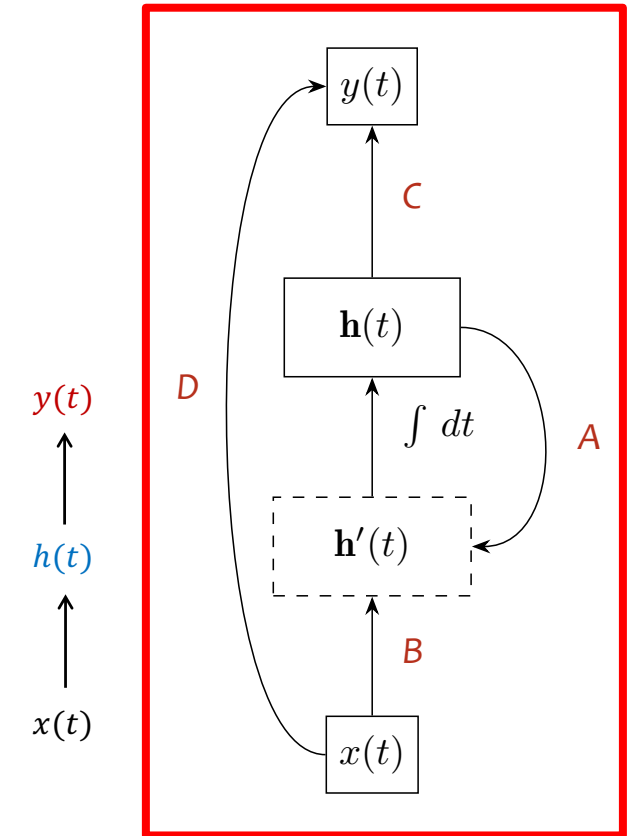
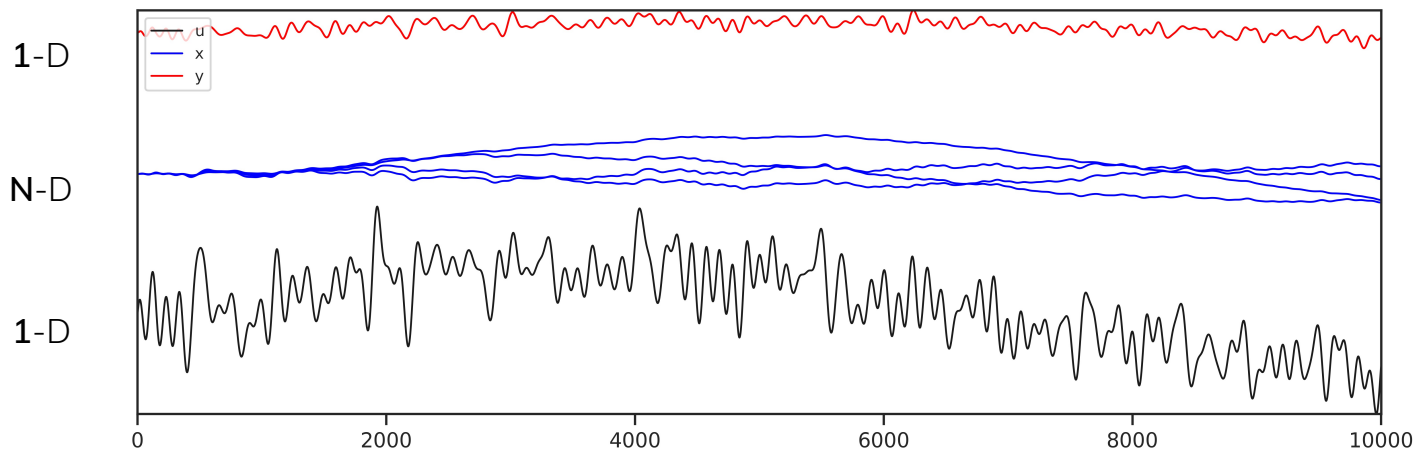
SSMs map 1D **function** to 1D **function**

SSM: 1D Continuous Representation

$$\frac{\partial \mathbf{h}(t)}{\partial t} = \mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t)$$

$$y(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}x(t)$$

$$\begin{aligned} x(t) &\in \mathbb{R} \\ y(t) &\in \mathbb{R} \\ \mathbf{h}(t) &\in \mathbb{R}^N \\ \mathbf{A} &\in \mathbb{R}^{N \times N} \\ \mathbf{B} &\in \mathbb{R}^{N \times 1} \\ \mathbf{C} &\in \mathbb{R}^{1 \times N} \\ \mathbf{D} &\in \mathbb{R}^{1 \times 1} \end{aligned}$$



SSM: 1D Discrete Recurrent Representation

Continuous Representation

- Uses parameters we will actually work with in the end
- Seamlessly represents any continuous $1D \rightarrow 1D$ function
- Impractical for real data

$$\begin{aligned}\mathbf{h}'(t) &= \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t) \\ y(t) &= \mathbf{C}\mathbf{h}(t) + \mathbf{D}x(t)\end{aligned}$$



Discrete Recurrent Representation

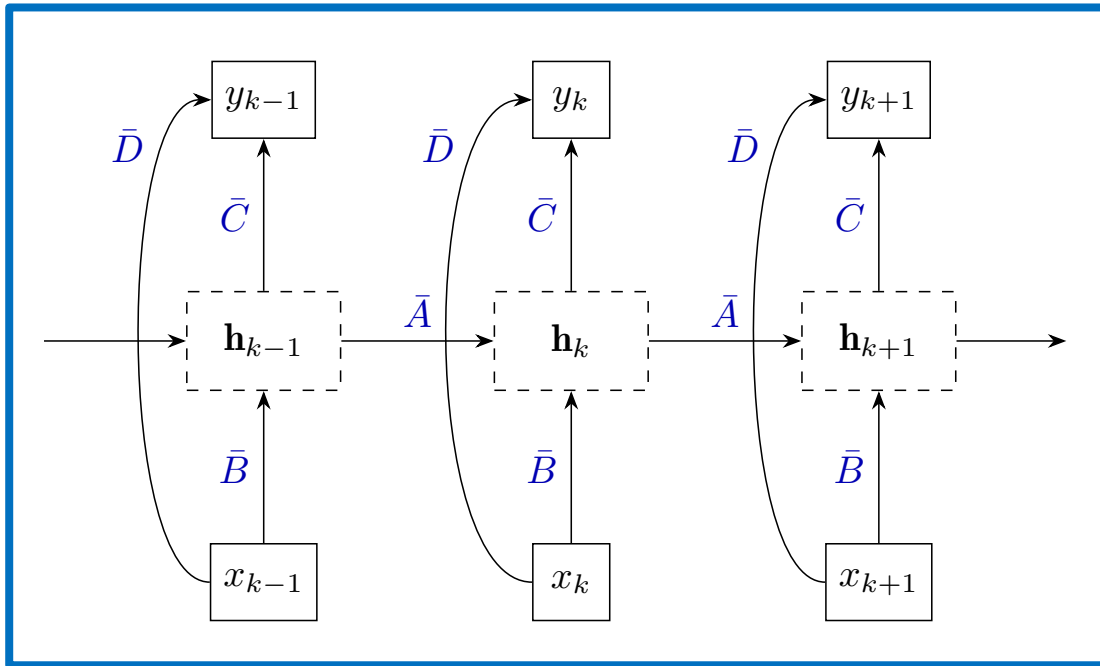
- A discrete approximation using different parameters which are **functions** of the original parameters A, B, C, D
- Allows us to work with real data

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}x_k$$

$$y_k = \bar{\mathbf{C}}\mathbf{h}_k + \bar{\mathbf{D}}x_k$$

SSM: 1D Discrete Recurrent Representation

Question: How can we depict this recurrent computation?



Discrete Recurrent Representation

- A discrete approximation using different parameters which are **functions** of the original parameters A, B, C, D
- Allows us to work with real data

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}x_k$$

$$y_k = \bar{\mathbf{C}}\mathbf{h}_k + \bar{\mathbf{D}}x_k$$

How to discretize a continuous SSM?

S4 uses a bilinear transformation to discretize the continuous SSM

$$\bar{\mathbf{A}} = e^{\Delta \mathbf{A}}$$

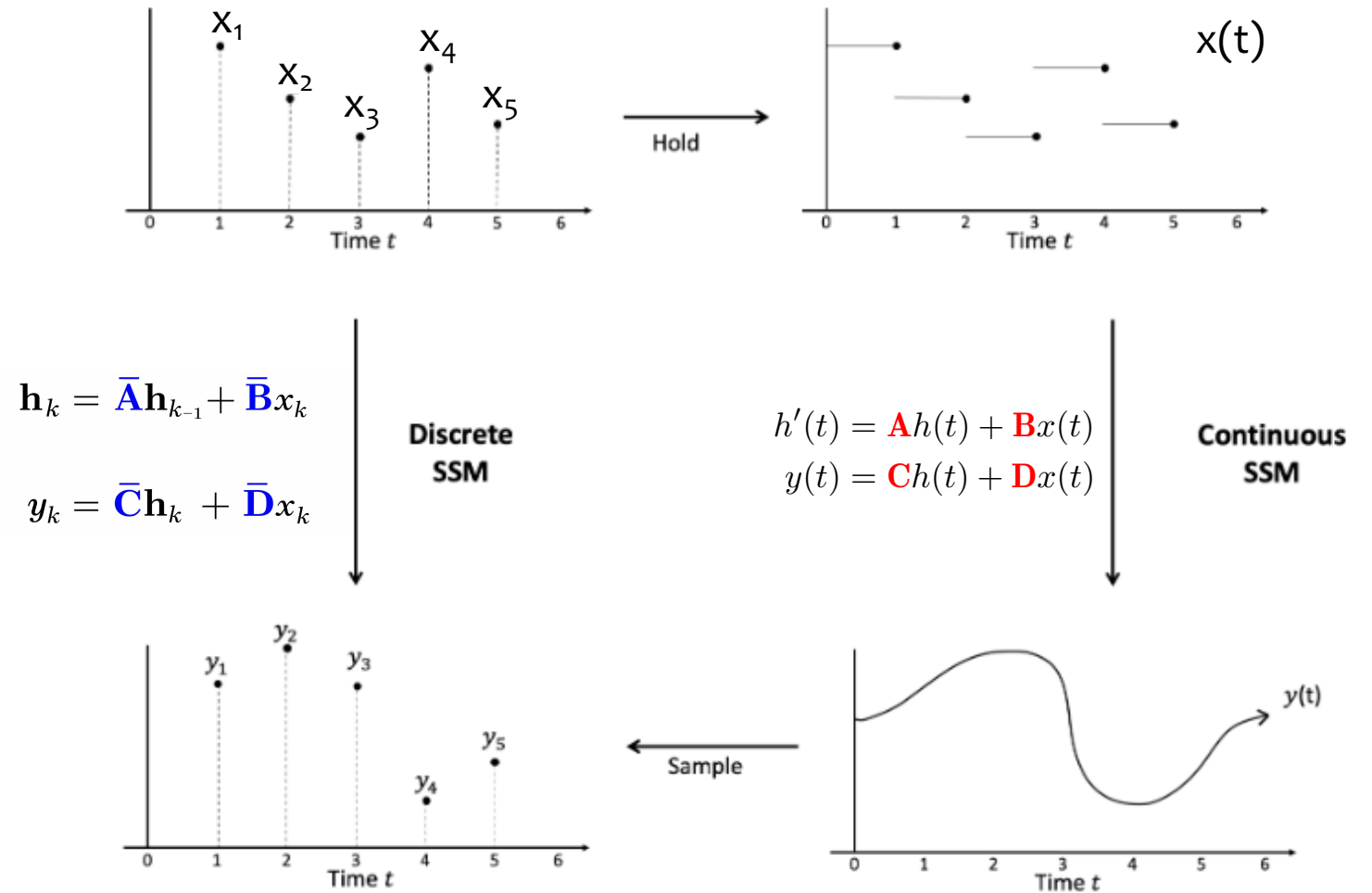
$$\bar{\mathbf{B}} = \mathbf{A}^{-1}(e^{\Delta \mathbf{A}} - \mathbf{I})\mathbf{B}$$

$$\bar{\mathbf{C}} = \mathbf{C}$$

$$\bar{\mathbf{D}} = \mathbf{D}$$

The bilinear transformation uses a first order Pade approximation:

$$e^x \approx \frac{1+x/2}{1-x/2}$$



The discrete-time SSM (left), a sequence-to-sequence map, is exactly equivalent to applying the

Figure from <https://hazyresearch.stanford.edu/blog/2022-01-14-s4-continuous-time-SSM> (right), a function-to-function map, on the held signal.

How to discretize a continuous SSM?

S4 uses a bilinear transformation to discretize the continuous SSM

$$\bar{\mathbf{A}} = e^{\Delta \mathbf{A}}$$

$$\bar{\mathbf{B}} = \mathbf{A}^{-1}(e^{\Delta \mathbf{A}} - \mathbf{I})\mathbf{B}$$

$$\bar{\mathbf{C}} = \mathbf{C}$$

$$\bar{\mathbf{D}} = \mathbf{D}$$

$$\bar{\mathbf{A}} = (\mathbf{I} - \frac{\Delta}{2} \cdot \mathbf{A})^{-1}(\mathbf{I} + \frac{\Delta}{2} \cdot \mathbf{A})$$

$$\bar{\mathbf{B}} = (\mathbf{I} - \frac{\Delta}{2} \cdot \mathbf{A})^{-1}\Delta \mathbf{B}$$

$$\bar{\mathbf{C}} = \mathbf{C}$$

$$\bar{\mathbf{D}} = \mathbf{D}$$

The bilinear transformation uses a first order Pade approximation:

$$e^x \approx \frac{1+x/2}{1-x/2}$$

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t)$$

$$y(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}x(t)$$



$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}x_k$$

$$y_k = \bar{\mathbf{C}}\mathbf{h}_k + \bar{\mathbf{D}}x_k$$

SSM: 1D Convolutional Representation

We unroll the recurrent computation as:
Assume a zero initial state: $h_{-1} = 0$.

$$h_0 = \bar{\mathbf{B}}x_0 \quad \mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}x_k$$

$$h_1 = \bar{\mathbf{A}}\bar{\mathbf{B}}x_0 + \bar{\mathbf{B}}x_1$$

$$h_2 = \bar{\mathbf{A}}^2\bar{\mathbf{B}}x_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}x_1 + \bar{\mathbf{B}}x_2$$

$\vdots$

$$y_0 = \bar{\mathbf{C}}\bar{\mathbf{B}}x_0 \quad y_k = \bar{\mathbf{C}}\mathbf{h}_k + \bar{\mathbf{D}}x_k$$

$$y_1 = \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}x_0 + \bar{\mathbf{C}}\bar{\mathbf{B}}x_1$$

Ignore for now,
can add later

$$y_2 = \bar{\mathbf{C}}\bar{\mathbf{A}}^2\bar{\mathbf{B}}x_0 + \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}x_1 + \bar{\mathbf{C}}\bar{\mathbf{B}}x_2$$

$\vdots$

$$y_k = \bar{\mathbf{C}}\bar{\mathbf{A}}^k\bar{\mathbf{B}}x_0 + \bar{\mathbf{C}}\bar{\mathbf{A}}^{k-1}\bar{\mathbf{B}}x_1 + \cdots + \bar{\mathbf{C}}\bar{\mathbf{A}}^1\bar{\mathbf{B}}x_{k-1} + \bar{\mathbf{C}}\bar{\mathbf{A}}^0\bar{\mathbf{B}}x_k$$

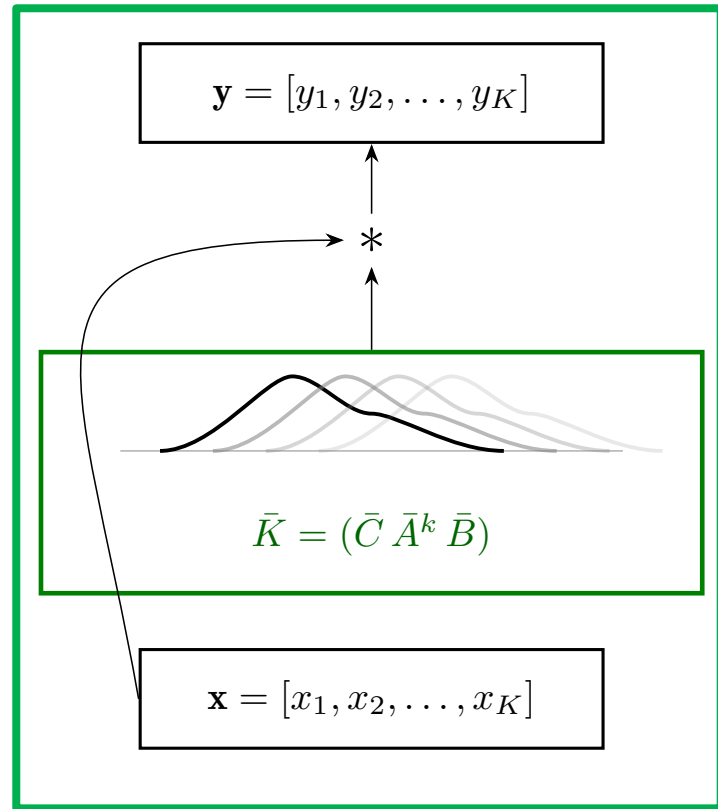
We can represent this as a *global* convolution computation:

$$\mathbf{y} = \bar{\mathbf{K}} * \mathbf{x}$$

where the SSM convolution kernel is:

$$\bar{\mathbf{K}} \in \mathbb{R}^L = (\bar{\mathbf{C}}\bar{\mathbf{A}}^0\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}^1\bar{\mathbf{B}}, \dots, \bar{\mathbf{C}}\bar{\mathbf{A}}^{L-2}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}})$$

SSM: 1D Convolutional Representation

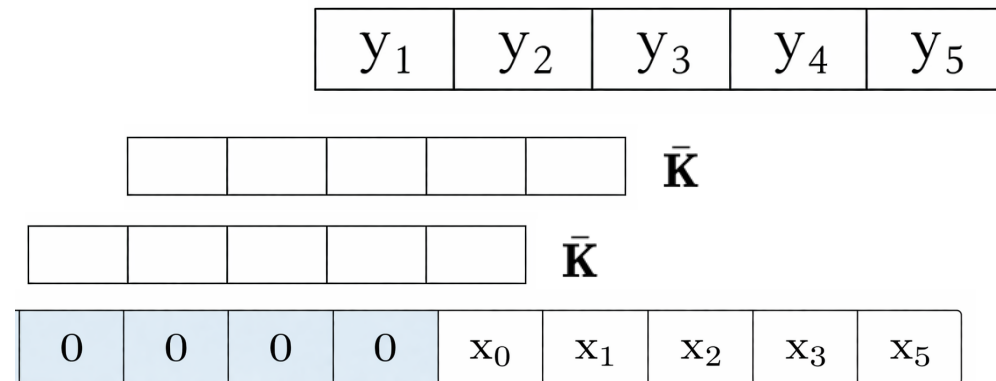


We can represent this as a *global* convolution computation:

$$\mathbf{y} = \bar{\mathbf{K}} * \mathbf{x}$$

where the SSM convolution kernel is:

$$\bar{\mathbf{K}} \in \mathbb{R}^L = (\bar{\mathbf{C}} \bar{\mathbf{A}}^0 \bar{\mathbf{B}}, \bar{\mathbf{C}} \bar{\mathbf{A}}^1 \bar{\mathbf{B}}, \dots, \bar{\mathbf{C}} \bar{\mathbf{A}}^{L-2} \bar{\mathbf{B}}, \bar{\mathbf{C}} \bar{\mathbf{A}}^{L-1} \bar{\mathbf{B}})$$



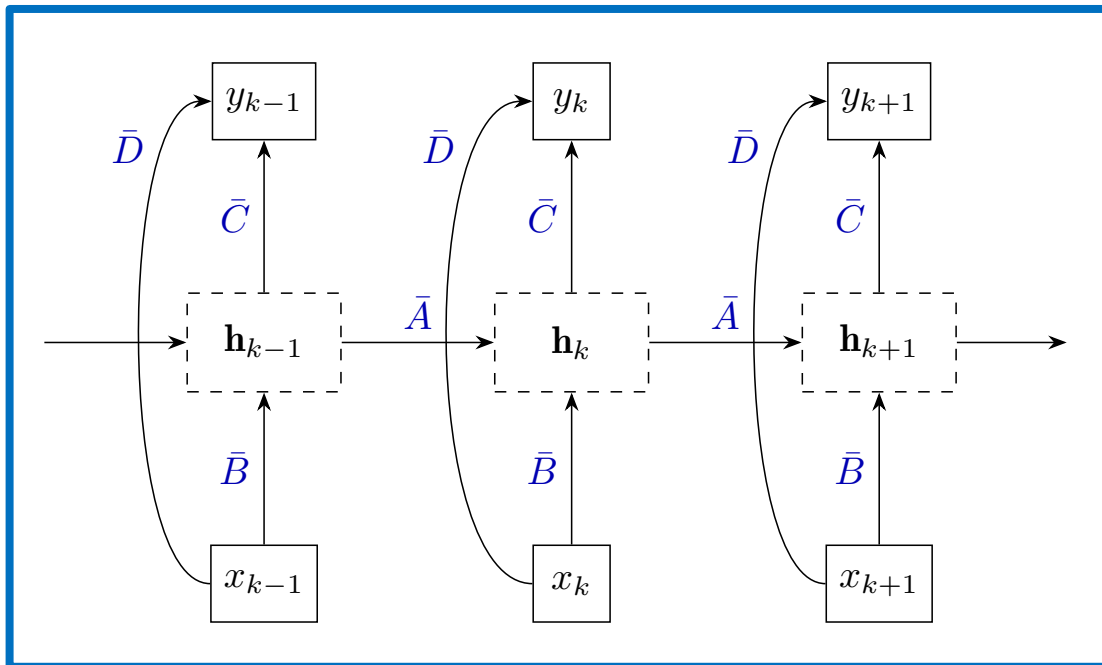
all y_k can be computed in parallel!

$$y_k = \bar{\mathbf{C}} \bar{\mathbf{A}}^k \bar{\mathbf{B}} x_0 + \bar{\mathbf{C}} \bar{\mathbf{A}}^{k-1} \bar{\mathbf{B}} x_1 + \dots + \bar{\mathbf{C}} \bar{\mathbf{A}}^1 \bar{\mathbf{B}} x_{k-1} + \bar{\mathbf{C}} \bar{\mathbf{A}}^0 \bar{\mathbf{B}} x_k$$

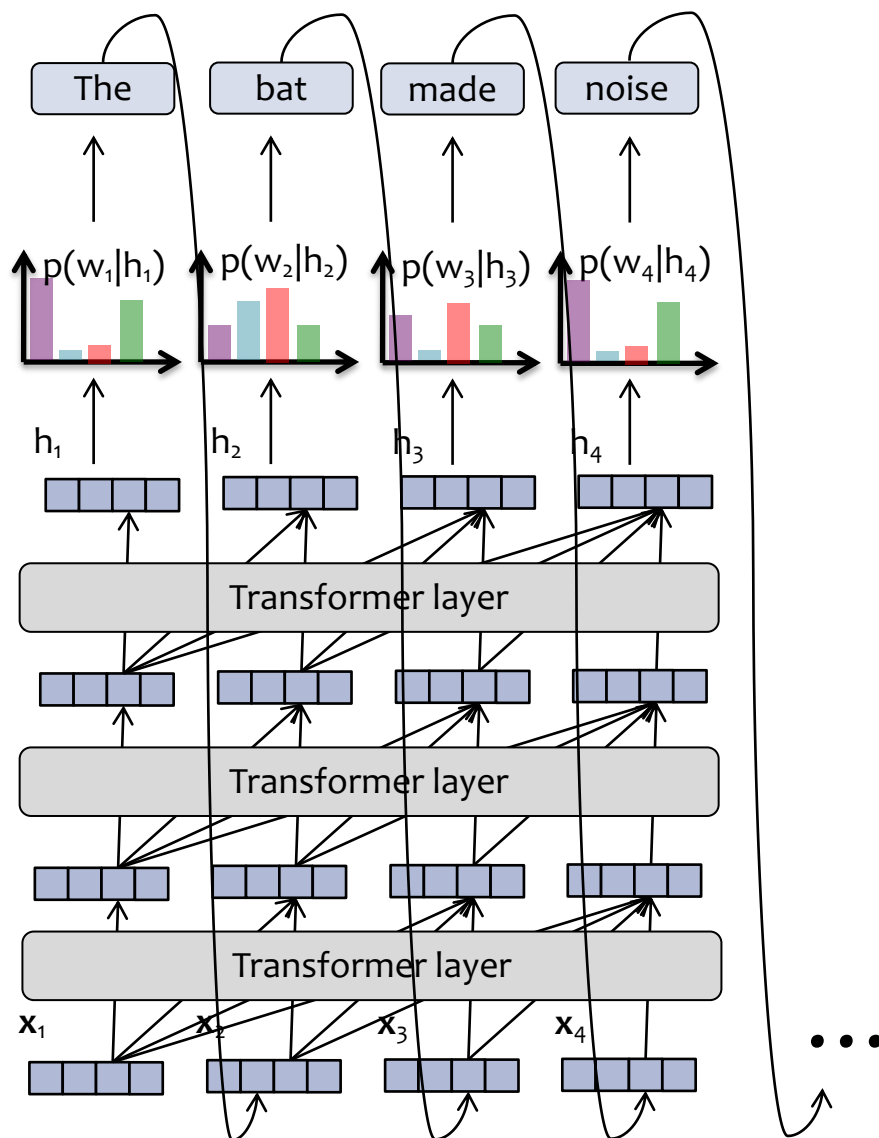
THE STRUCTURED STATE SPACE SEQUENCE MODEL (S4)

SSM as a Neural Network Layer

- We can take H copies of the 1D recurrent representation
- Let each copy have its own parameters
- This is just like multiple (indep.) heads in Attention
- And just like multiple (indep.) channels in Convolution
- So we get...



Transformer Language Model



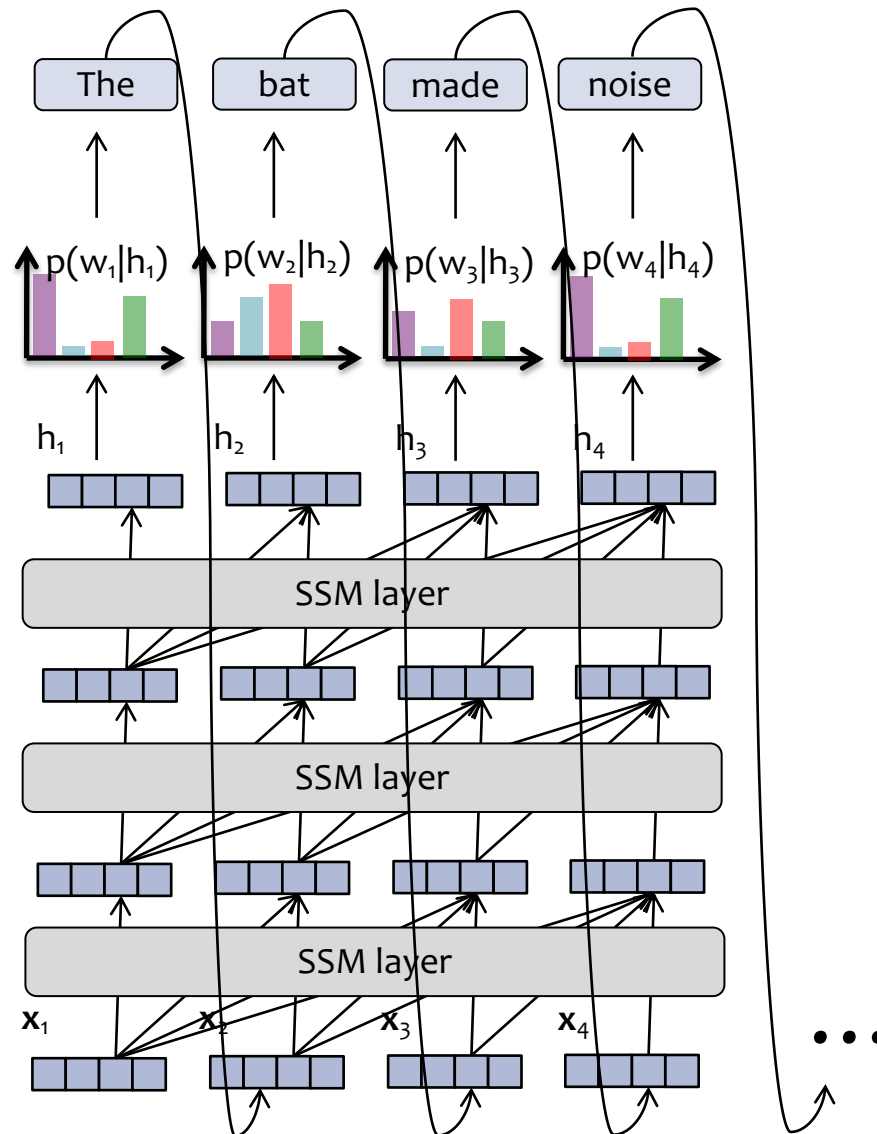
Each layer of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

The language model part is just like an RNN-LM.

SSM inside a Deep Language Model



Each layer of an S4 LM consists of several **sublayers as well** including an SSM, nonlinearity, etc.

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

The language model part is just like an RNN-LM or Transformer-LM

Efficiency of SSM, RNN, & Attention

For SSMs:

1. At test time, generation does NOT need a KV-cache in our **Recurrent representation**, so we can effortlessly generate truly long sequences (unlike Transformers, but just like RNNs)
2. At train time, we can use the **Convolution representation** to do fast parallel training (just like Transformers, but unlike RNNs)

	Train	Test
Recurrence	slow	fast
Attention	fast	slow
SSM	fast	fast

S4 Model

We need several additional tricks to get training to work well:

- HiPPO Matrix
 - we initialize the matrix A very carefully
- Efficient computation
 - we decompose A so that we can compute the kernel K very efficiently and in a numerically stable way

$$\bar{\mathbf{K}} \in \mathbb{R}^L = (\bar{\mathbf{C}}\bar{\mathbf{A}}^0\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}^1\bar{\mathbf{B}}, \dots, \bar{\mathbf{C}}\bar{\mathbf{A}}^{L-2}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}})$$

Numerical instability can be especially pronounced here!

Selective State Space Model

with Hardware-aware State Expansion

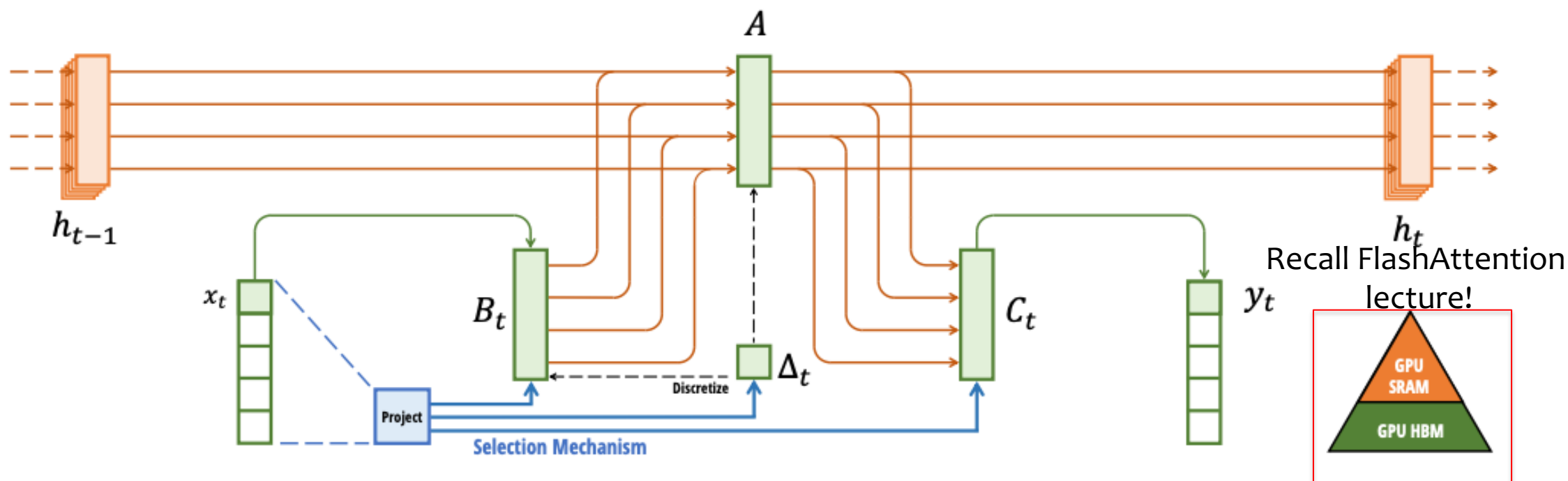
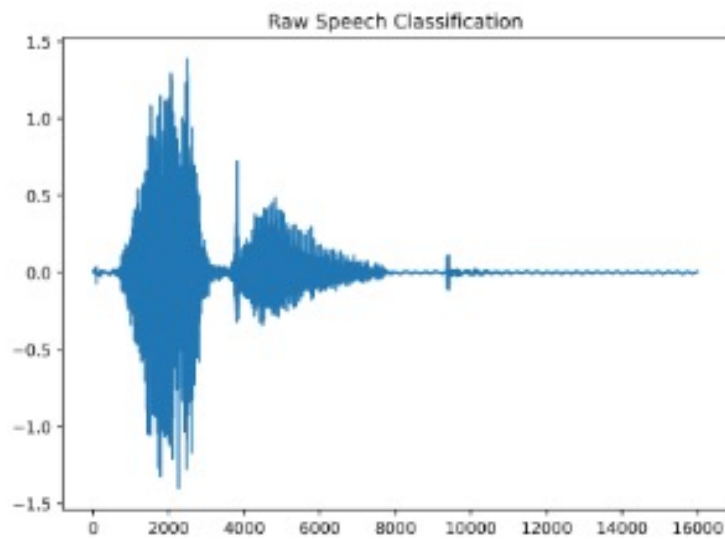


Figure 1: (**Overview.**) Structured SSMs independently map each channel (e.g. $D = 5$) of an input x to output y through a higher dimensional latent state h (e.g. $N = 4$). Prior SSMs avoid materializing this large effective state (DN , times batch size B and sequence length L) through clever alternate computation paths requiring time-invariance: the (Δ, A, B, C) parameters are constant across time. Our selection mechanism adds back input-dependent dynamics, which also requires a careful hardware-aware algorithm to only materialize the expanded states in more efficient levels of the GPU memory hierarchy.

S4 Results: Train and test on different input granularities



		Train: 16K Hz	Test: 8K Hz
	MFCC	RAW	0.5×
Transformer	90.75	✗	✗
Performer	80.85	30.77	30.68
ODE-RNN	65.9	✗	✗
NRDE	89.8	16.49	15.12
ExpRNN	82.13	11.6	10.8
LipschitzRNN	88.38	✗	✗
CKConv	95.3	71.66	<u>65.96</u>
WaveGAN-D	✗	<u>96.25</u>	✗
LSSL	93.58	✗	✗
S4	<u>93.96</u>	98.32	96.30

MAMBA

Selective State Space Models

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

- 1: $A : (D, N) \leftarrow \text{Parameter}$
▷ Represents structured $N \times N$ matrix
- 2: $B : (D, N) \leftarrow \text{Parameter}$
- 3: $C : (D, N) \leftarrow \text{Parameter}$
- 4: $\Delta : (D) \leftarrow \tau_\Delta(\text{Parameter})$
- 5: $\bar{A}, \bar{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$
- 6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$
▷ Time-invariant: recurrence or convolution

7: **return** y

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}_k x_k$$

$$y_k = \bar{\mathbf{C}}_k \mathbf{h}_k + \bar{\mathbf{D}} x_k$$

Algorithm 2 SSM + Selection (S6)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

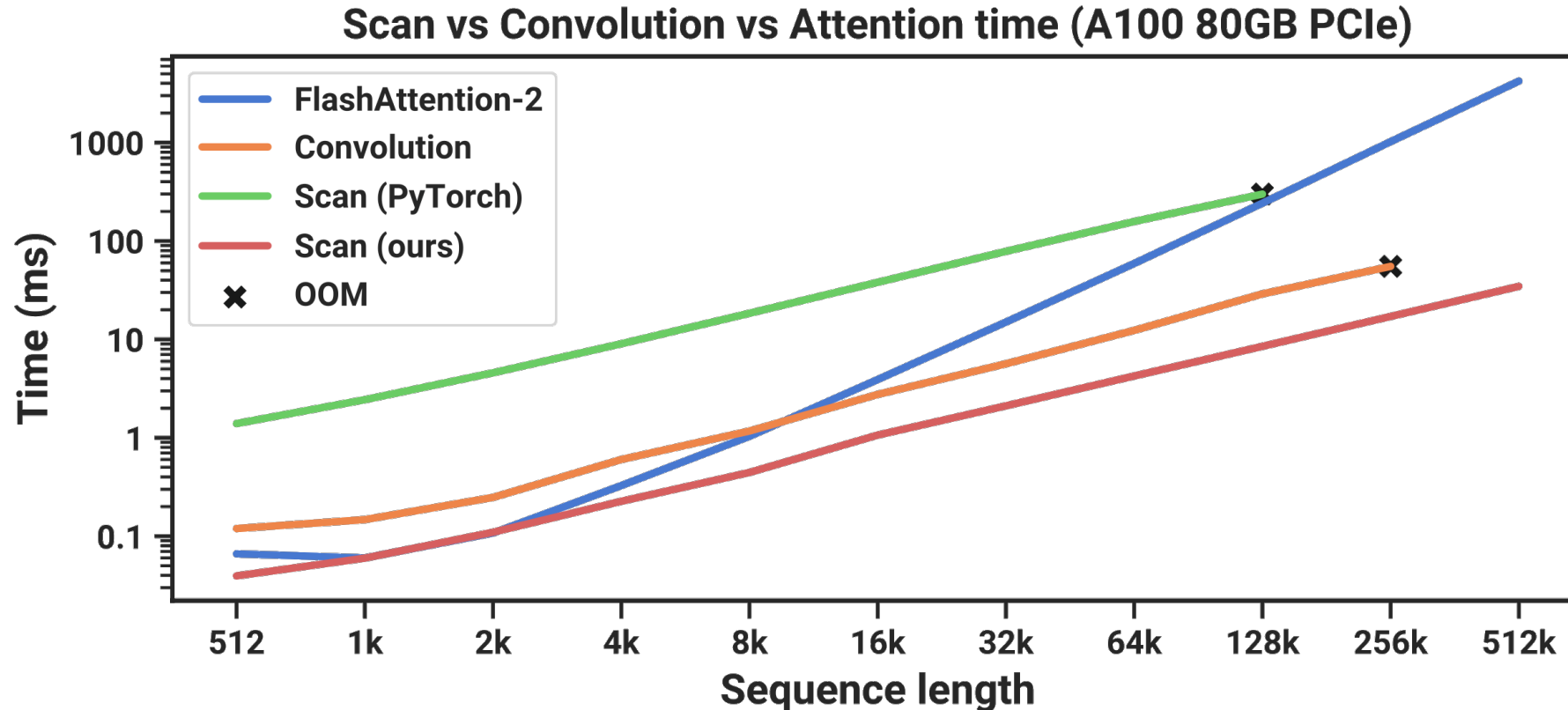
- 1: $A : (D, N) \leftarrow \text{Parameter}$
▷ Represents structured $N \times N$ matrix
- 2: $B : (B, L, N) \leftarrow s_B(x)$
- 3: $C : (B, L, N) \leftarrow s_C(x)$
- 4: $\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
- 5: $\bar{A}, \bar{B} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, A, B)$
- 6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$
▷ **Time-varying**: recurrence (*scan*) only

7: **return** y

- **Selective** state space models differ from S4 in that they let the parameters B and C vary at each timestep

Mamba's Scan Implementation

- We can no longer compute the kernel K once up front
- Instead we perform an efficient scan implementation



Mamba Results

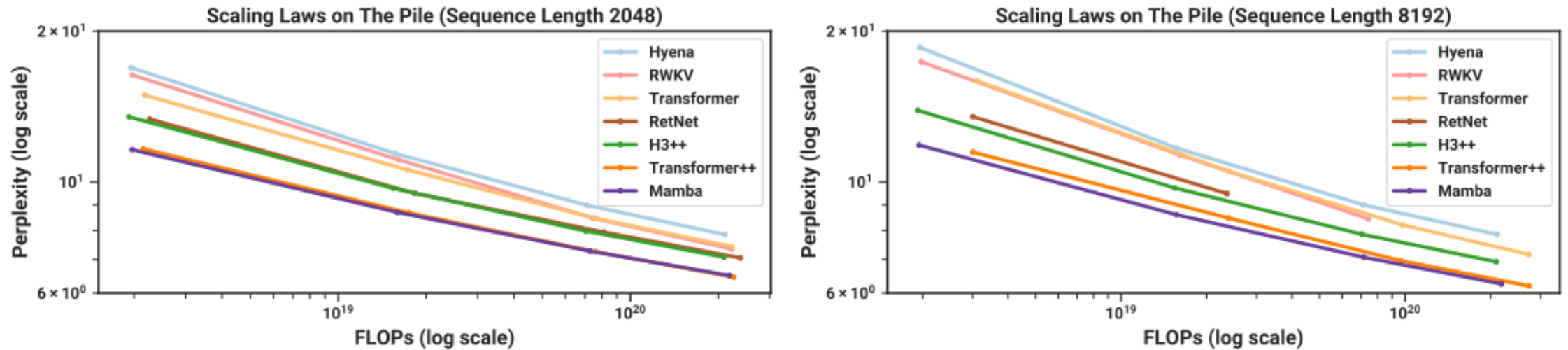
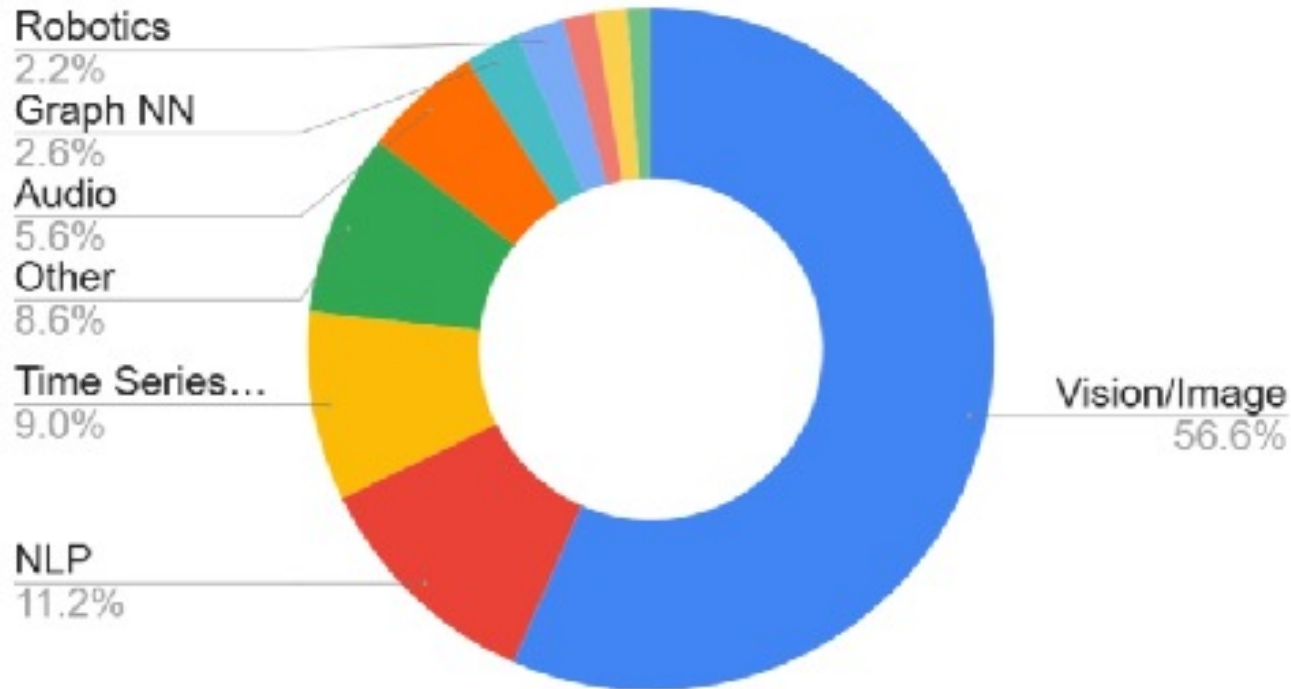


Figure 4: (**Scaling Laws.**) Models of size $\approx 125M$ to $\approx 1.3B$ parameters, trained on the Pile. Mamba scales better than all other attention-free models and is the first to match the performance of a very strong “Transformer++” recipe that has now become standard, particularly as the sequence length grows.

- Main takeaway: Mamba was the first non-attention based LM to challenge a Transformer

Mamba Use in the Real World

Mamba Paper Categories - 267 papers up till June 27th



Strong out-of-the-box
on **general modalities**
(not just language!)

LINEAR ATTENTION AND SSMS

Linear Attention

- Linear attention is identical to standard attention, but drops the softmax: (below M is the causal mask)

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V} \in \mathbb{R}^{L \times d_v} \qquad \mathbf{o}_t = \sum_{i=1}^t (\mathbf{v}_i \mathbf{k}_i^\top) \mathbf{q}_t = \sum_{i=1}^t \mathbf{v}_i (\mathbf{k}_i^\top \mathbf{q}_t) \in \mathbb{R}^{d_v},$$

- And can be expressed as a recurrence:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \in \mathbb{R}^{d_v \times d_k}, \qquad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \in \mathbb{R}^{d_v}$$

- Draws a direct connection between Transformers and SSMs

Linear Attention

- Linear attention is identical to standard attention, but drops the softmax: (below M is the causal mask)

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V} \in \mathbb{R}^{L \times d_v} \qquad \mathbf{o}_t = \sum_{i=1}^t (\mathbf{v}_i \mathbf{k}_i^\top) \mathbf{q}_t = \sum_{i=1}^t \mathbf{v}_i (\mathbf{k}_i^\top \mathbf{q}_t) \in \mathbb{R}^{d_v},$$

- And can be expressed as a recurrence:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \in \mathbb{R}^{d_v \times d_k}, \qquad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \in \mathbb{R}^{d_v}$$

- Various forms of linear attention have been proposed:

Method	Online Update
LA	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$
Mamba2	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$
Longhorn	$\mathbf{S}_t = \mathbf{S}_{t-1} (\mathbf{I} - \epsilon_t \mathbf{k}_t \mathbf{k}_t^\top) + \epsilon_t \mathbf{v}_t \mathbf{k}_t^\top, \epsilon_t = \frac{\beta_t}{1 + \beta_t \mathbf{k}_t^\top \mathbf{k}_t}$
DeltaNet	$\mathbf{S}_t = \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$
Gated DeltaNet	$\mathbf{S}_t = \mathbf{S}_{t-1} \left(\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \right) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$

HYBRID MODELS

Hybrid Models

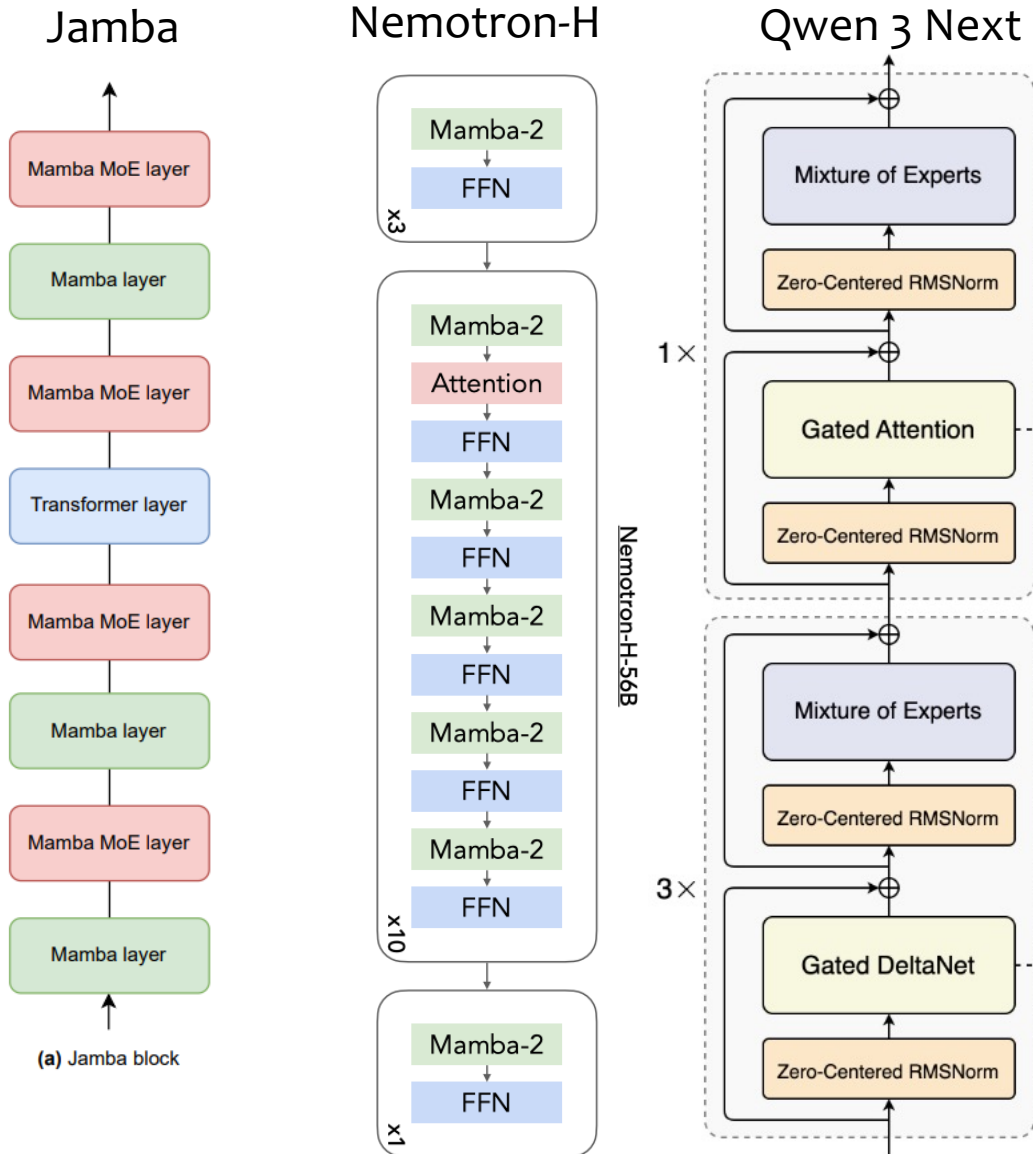
Hybrid models combine the best of Transformers and SSMs

Motivation:

- *Long-context scalability:* Reduce the quadratic cost of self-attention while maintaining or improving modeling ability for sequences spanning tens or hundreds of thousands of tokens.
- *Better hardware utilization:* Exploit architectures that stream activations sequentially (like SSMs) to improve throughput and fit larger contexts on fixed-memory GPUs.
- *Robust generalization:* Mix inductive biases—e.g., attention for global dependencies, convolutional or state-space layers for local and temporal structure—to handle diverse data types (text, audio, vision).

Examples: Jamba, Nemotron-H, Qwen 3 Next

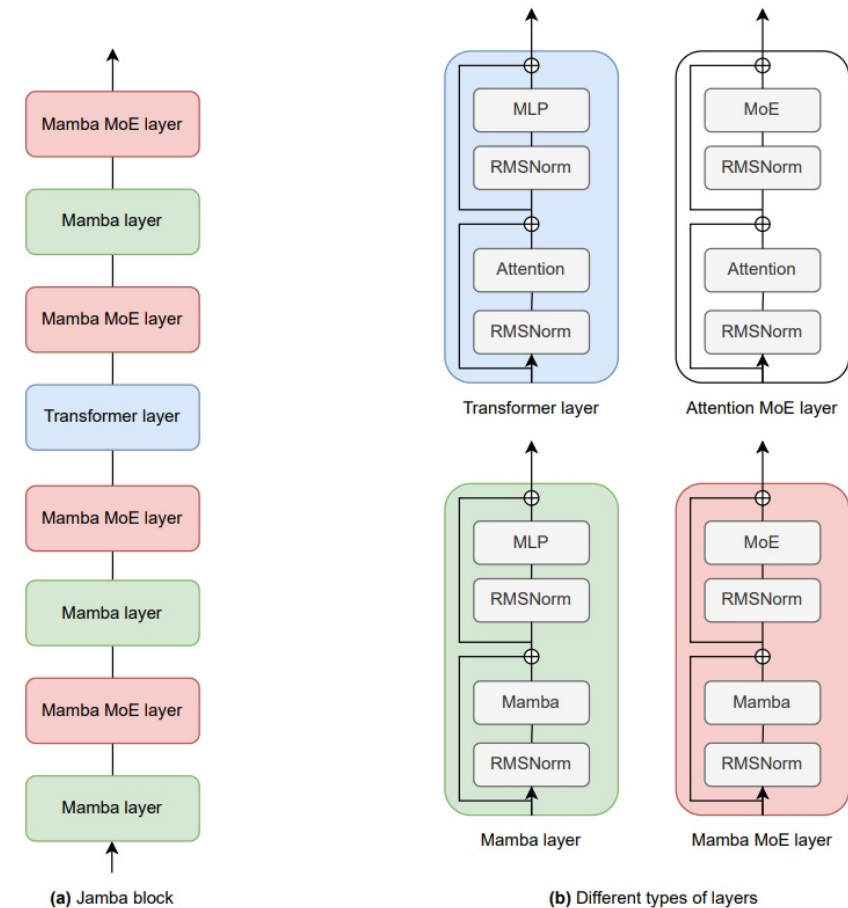
Hybrid Models



- Common to all these models: standard attention layers are **interspersed** with linear attention layers
- The standard attention has **quadratic complexity**, and the linear attention has **linear complexity** in the sequence length
- **Goal:** reduce memory requirements and speed up generation (all of these models accomplish this)

Jamba

- Jamba was the **first** hybrid model to combine Transformer layers with SSMs layers (2024)
- The architecture dramatically **reduces the size of the KV cache** for long contexts
- Jamba enables dramatically **longer contexts** to fit on a single GPU than traditional Transformer LMs
- This enables much **higher throughput** (measured in tokens per second) at generation time

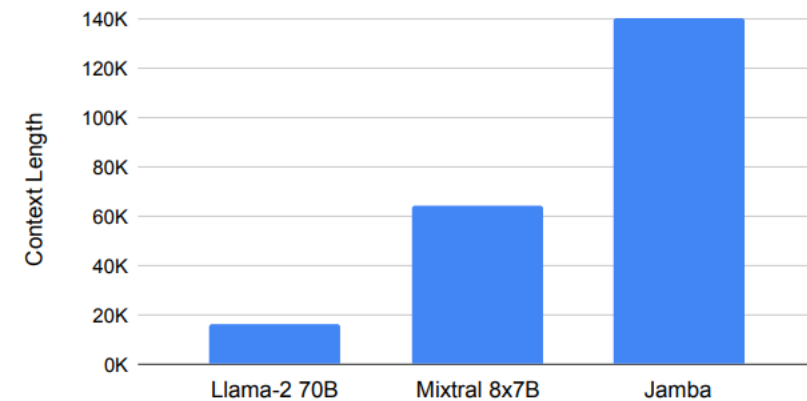


	Available params	Active params	KV cache (256K context, 16bit)
LLAMA-2	6.7B	6.7B	128GB
Mistral	7.2B	7.2B	32GB
Mixtral	46.7B	12.9B	32GB
Jamba	52B	12B	4GB

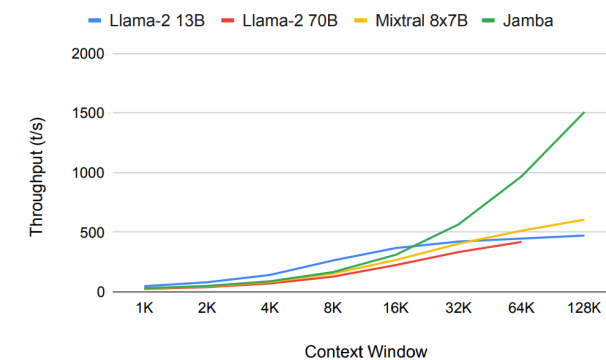
Jamba

- Jamba was the **first** hybrid model to combine Transformer layers with SSMs layers
- The architecture dramatically **reduces the size of the KV cache** for long contexts
- Jamba enables dramatically **longer contexts** to fit on a single GPU than traditional Transformer LMs
- This enables much **higher throughput** (measured in tokens per second) at generation time

Context length fitting a single 80GB A100 GPU



Throughput (4 A100 GPUs)



(b) Throughput at different context lengths (single batch, 4 A100 GPUs). With a context of 128K tokens, Jamba obtains 3x the throughput of Mixtral, while Llama-2-70B does not fit with this long context.

Nemotron-H

- Nemotron-H demonstrated that performance of a standard Transformer (Nemotron-T) could be retained while gaining dramatic throughput improvements
- Also introduced a vision-language model (VLM) variant

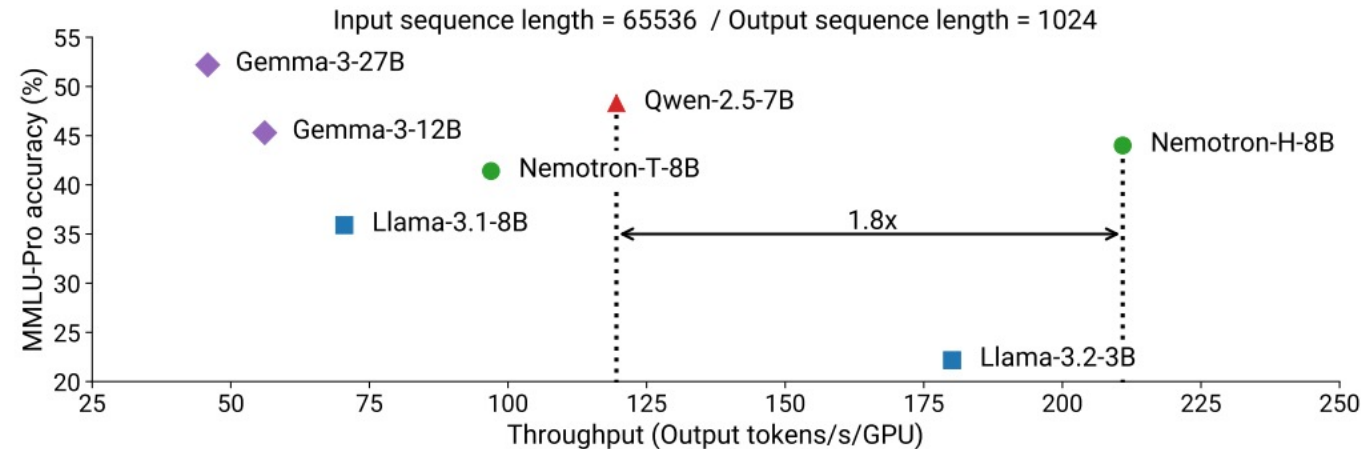
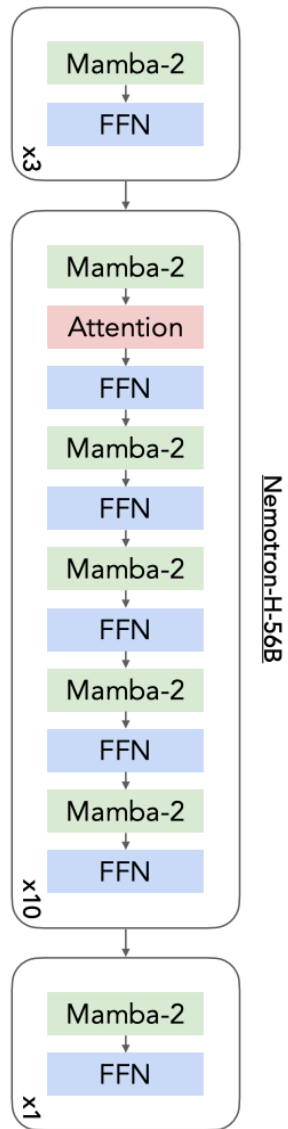
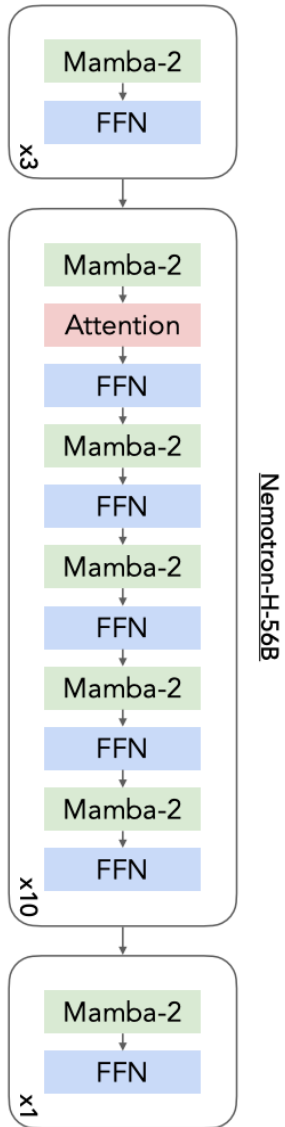


Figure 7 | MMLU-Pro accuracy versus inference throughput (normalized by number of GPUs used) for Nemotron-H-8B-Base compared to existing similarly-sized Transformer models.

Nemotron-H

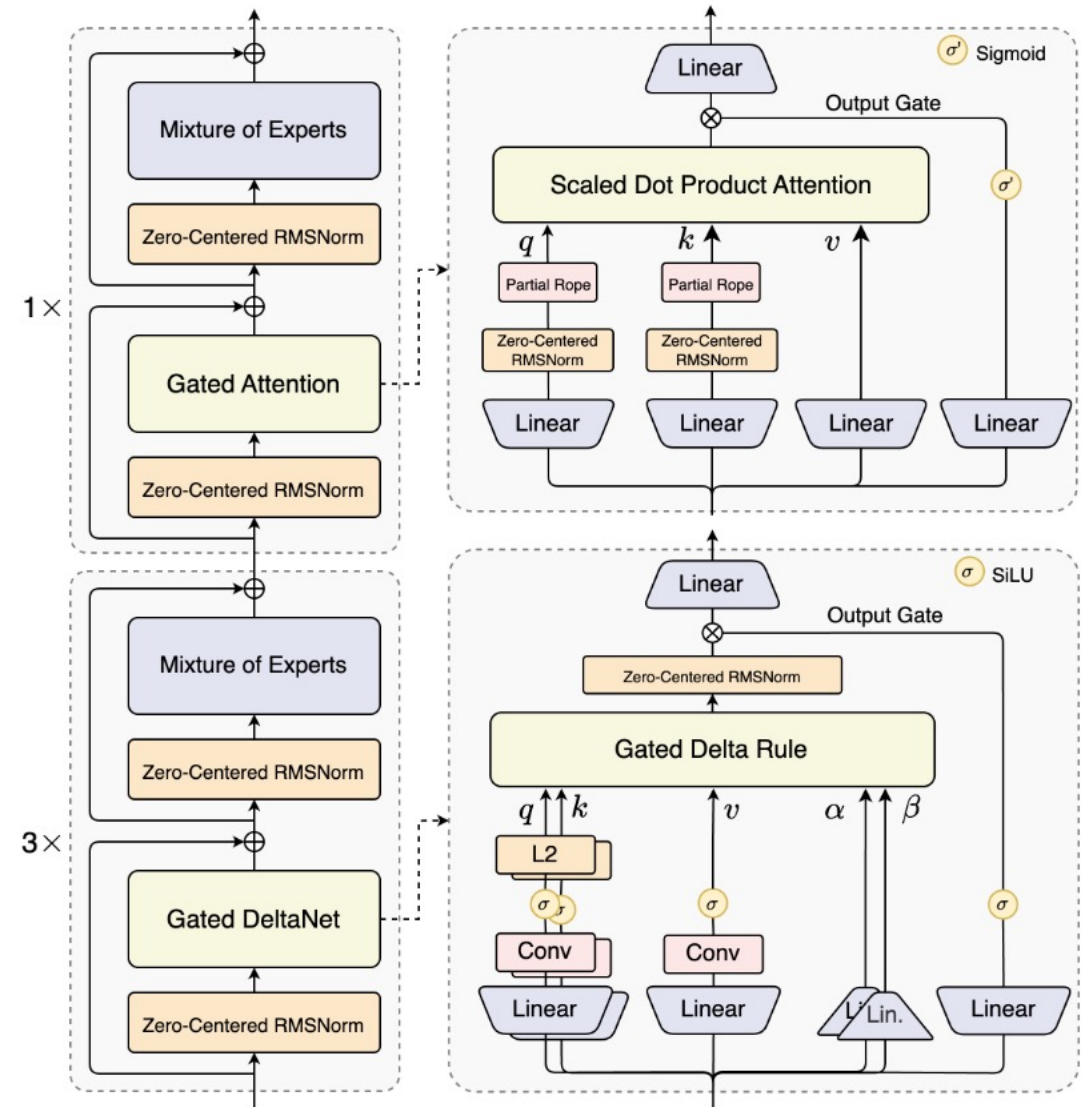


- Nemotron-H demonstrated that performance of a standard Transformer (Nemotron-T) could be retained while gaining dramatic throughput improvements
- Also introduced a vision-language model (VLM) variant

Task	Nemotron-H 56B-VLM	VLM w/ Qwen2.5 72B-Instruct	NVLM-D-1.0 72B (2024-09-17)
MMMU (val)	63.6	65.1	62.6 [†]
MathVista	70.7	70.5	66.7 [†]
ChartQA	89.4	88.9	86.0
AI2D	94.7	94.9	94.2
OCRBench	862	869	853
TextVQA	81.1	83.5	82.1
RealWorldQA	68.4	71.4	69.7
DocVQA	93.2	92.0	92.6

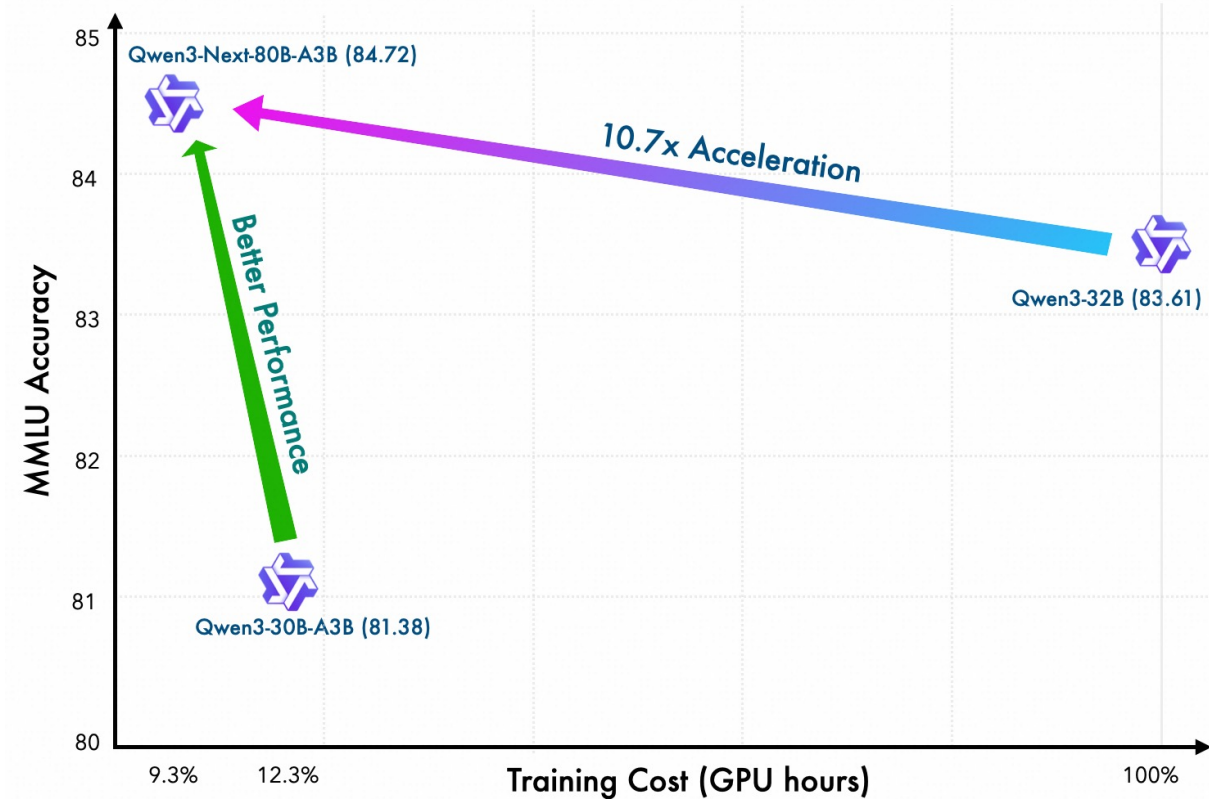
Qwen-3-Next

- Qwen-3-Next uses Gated Delta Net layers instead of standard linear attention layers



Qwen-3-Next

- Qwen-3-Next uses Gated Delta Net layers instead of standard linear attention layers
- Both training time and generation are dramatically reduced, compared to dense models of comparable size



Qwen-3-Next

- Qwen-3-Next uses Gated Delta Net layers instead of standard linear attention layers
- Both training time and generation are dramatically reduced, compared to dense models of comparable size
- Impressive performance across standard benchmarks

	Qwen3-30B-A3B	Qwen3-32B	Qwen3-Next-80B-A3B	Qwen3-235B-A22B
	Base	Base	Base	Base
Architecture	MoE	Dense	MoE	MoE
# Total Params	30B	32B	80B	235B
# Activated Params	3B	32B	3B	22B
<i>General Tasks</i>				
MMLU	81.38	83.61	<u>84.72</u>	87.81
MMLU-Redux	81.17	83.41	<u>83.80</u>	87.40
MMLU-Pro	61.49	65.54	<u>66.05</u>	68.18
SuperGPQA	35.72	39.78	<u>41.52</u>	44.06
BBH	81.54	<u>87.38</u>	87.13	88.87
<i>Math, STEM & Coding Tasks</i>				
GPQA	43.94	49.49	43.43	<u>47.47</u>
GSM8K	91.81	<u>93.40</u>	90.30	94.39
MATH	59.04	61.62	<u>62.36</u>	71.84
EvalPlus	71.45	72.05	<u>72.89</u>	77.60
CRUX-O	67.20	72.50	<u>74.25</u>	79.00
<i>Multilingual Tasks</i>				
MGSM	79.11	83.06	81.28	83.53
MMMLU	81.46	83.83	<u>84.43</u>	86.70
INCLUDE	67.00	67.87	<u>69.79</u>	73.46

Motivation

- <https://www.isattentionallyouneed.com/>

Is Attention All You Need?



Current Status: Yes

Time Remaining: 631d 22h 55m 11s

Proposition:

On January 1, 2027, a Transformer-like model will continue to hold the state-of-the-art position in most benchmarked tasks in natural language processing.